

КУРСОВАЯ РАБОТА

по дисциплине Вычислительная математика
Вариант 8А

Выполнил:

Студент

Гр. 5130904/30007

А. Б. Семенов

Задача:

Задание N 08.

Расчет вертикального прогиба балки.

Рассмотрим консольную балку с переменным поперечным сечением, показанную на рисунке.

Длина балки- ℓ , в ее конце-нагрузка P .

x - горизонтальное расстояние вдоль балки,

y - вертикальный прогиб.

Дифференциальное уравнение для вертикального прогиба имеет вид:

$$\frac{d^4 y}{dx^4} = -\frac{2}{I} \frac{dI}{dx} \cdot \frac{d^3 y}{dx^3} - \frac{1}{I} \frac{d^2 I}{dx^2} \cdot \frac{d^2 y}{dx^2} + \frac{\omega}{EI},$$

здесь M - изгибающий момент,

E - модуль Юнга,

I - момент инерции,

P -нагрузка на балку.

$$\omega = -\frac{d^2 M}{dx^2},$$

$$M(x) = -P(\ell - x),$$

$$I(x) = 5 \left(1 + 4e^{-\frac{6x}{\ell}} \right),$$

$$E = 3 \cdot 10^7,$$

$$y(0) = y'(0) = 0,$$

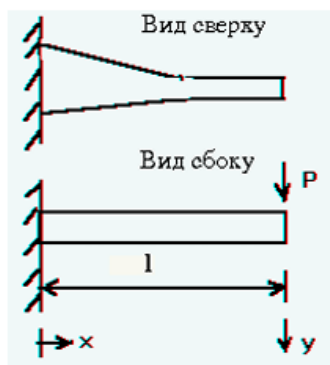
$$y''(0) = \frac{P\ell}{75} \cdot 10^{-7},$$

$$y'''(0) = \frac{P \cdot 3,8}{75} \cdot 10^{-7}.$$

Вычислить $y(\ell)$ для различных значений P : $500 \leq P \leq 1000$ с шагом 100.

Построить сплайн по значениям $y(\ell)$ в зависимости от P и с его помощью оценить $y(\ell)$ для $P=750$. Значение ℓ задается преподавателем.

Оценить погрешность результата и влияние на точность погрешности исходных данных.



Дополнительные условия:

Вариант N 8A.

$$\ell = 100 \cdot x^* - \text{где } x^* - \text{корень уравнения } \int_0^{20} \frac{e^{-xz}}{x+1} dz = 0.59639442 \cdot x \quad \text{на промежутке } [0,8, 1,1].$$

Код:

```
import numpy as np
from scipy.integrate import quad, solve_ivp
from scipy.optimize import fsolve
from scipy.interpolate import CubicSpline
import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt

def integral_eqn(x):
    integrand = lambda z: np.exp(-x * z) / (x + 1)
    integral, _ = quad(integrand, 0, 20)
    return integral - 0.59639442 * x

x_star = fsolve(integral_eqn, 1.0)[0]
l = 100 * x_star
E = 3e7
P_values = np.arange(500, 1100, 100)

print(f"Найденный корень x* = {x_star:.6f}")
print(f"Длина балки l = {l:.2f}")

def beam_equation(x, y, P):
    I = 5 * (1 + 4 * np.exp(-6 * x / l))
    dI = -120 / l * np.exp(-6 * x / l)
    d2I = 720 / (l ** 2) * np.exp(-6 * x / l)

    omega: int = 0 # d^2M/dx^2 = 0 для линейного момента

    dydx = np.zeros_like(y)
    dydx[0] = y[1]
    dydx[1] = y[2]
    dydx[2] = y[3]

    dydx[3] = - (2 / I) * dI * y[3] - (1 / I) * d2I * y[2] + omega / (E * I
** 2)

    return dydx

y_l_values = []

for P in P_values:
    I0 = 5 * (1 + 4) # при x=0 (e^0 = 1)
    y0 = [0,
0,
(P * l)/75 * 1e-7,
(P * 3.8)/75 * 1e-7]

    sol = solve_ivp(beam_equation, [0, l], y0, args=(P,),
method='RK45', rtol=1e-6, atol=1e-8)
    y_l_values.append(sol.y[0, -1])

cs = CubicSpline(P_values, y_l_values)
P_750 = 750
y_750 = cs(P_750)

h = P_values[1] - P_values[0]
```

```

error_estimate = h ** 4 * np.max(np.abs(cs(P_values, 4))) / 384

print("\nРезультаты расчета:")
print("P | y(l)")
for P, y in zip(P_values, y_l_values):
    print(f"{P:5d} | {y:.6f}")
print(f"\nИнтерполированное значение при P=750: {y_750:.6f} м")
print(f"Оценка погрешности: {error_estimate:.2e}")

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(P_values, y_l_values, 'o', label='Расчетные точки')
P_interp = np.linspace(500, 1000, 100)
plt.plot(P_interp, cs(P_interp), label='Кубический сплайн')
plt.axvline(x=750, color='r', linestyle='--', label='P=750 Н')
plt.xlabel('Нагрузка P')
plt.ylabel('Прогиб y(l)')
plt.title('Зависимость прогиба от нагрузки')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
sol_750 = solve_ivp(beam_equation, [0, 1], y0, args=(750,),
    method='RK45', dense_output=True)
x_plot = np.linspace(0, 1, 100)
y_plot = sol_750.sol(x_plot)[0]
plt.plot(x_plot, y_plot)
plt.xlabel('Координата x')
plt.ylabel('Прогиб y(x)')
plt.title(f'Форма прогиба при P=750 Н (l={l:.2f} м)')
plt.grid(True)

plt.tight_layout()
plt.show()

```

Объяснение:

Использованные библиотеки:

- numpy (для численных расчетов)
- scipy (для численных расчетов)
- Matplotlib (для визуализации, использован бэкенд 'TkAgg')

Методы и переменные:

integral_eqn(x)

Вычисляет 1 из Дополнительного условия к задаче 8А

$$\int_0^{20} \frac{e^{-xz}}{x+1} dz = 0.59639442 \cdot x$$

integrand – лямбда функция относительно x с переменной интегрирования z, которая возвращает:

$$\frac{e^{-xz}}{x+1}$$

Integral – решение интеграла, принимающего integrand и нижнюю и верхнюю границы интегрирования соответственно 0 и 20.

_ – оценка абсолютной погрешности, которую в коде мы не используем.

x_star = fsolve(integral_eqn, 1.0)[0]

x_star – один из множителей для нахождения l, длины балки.

fsolve – уточняет значение x с каждой итерацией начиная с 1.0, пока не найдет x*, при котором integral_eqn(x) ≈ 0

P_values = np.arange(500, 1100, 100)

Значения P с шагом 100 от 500 до 1000

beam_equation(x, y, P)

На входе:

x – горизонтальное расстояние вдоль балки,

y – вертикальный прогиб,

P – нагрузка на балку;

Метод определяет систему дифференциальных уравнений 4-го порядка, вычисляет момент инерции I и его производные.

Для упрощения примем omega = 0, так как вторая производная момента равно нулю.

Решение:

Создаем пустой массив `y_l_values = []`.

Для каждого значения P находим:

Уточнение: так как мы ищем производные в точке $x = 0$, то нахождение переменных упрощается.

- $I0$ – момент инерции.
- $Y0$ – начальные условия с производными, зависящими от P .
- `sol` – решение системы ОДУ, где дополнительные аргументы:
 - `[0, 1]` – интервал интегрирования от начала балки 0 до её конца 1 (длина балки).
 - `Args = (P,)` – дополнительные аргументы для `beam_quation`,
 - `method='RK45'` – метод решения Рунге-Кутты 4, 5 порядка,
 - `rtol=1e-6` – относительная погрешность,
 - `atol=1e-8` – абсолютная погрешность.

```
y_l_values.append(sol.y[0, -1])
```

- `sol.y[0, -1]` – двумерный массив
 - 0 – значение прогиба $y(x)$ на всем интервале
 - -1 – последний элемент массива, прогиб в точке $x = 1$ (длина балки)
- `Append` добавляет значение `sol.y[0, -1]` в массив `y_l_values`.

```
cs = CubicSpline(P_values, y_l_values)
```

Строим кубический сплайн по расчетным точкам.

```
y_750 = cs(P_750)
```

Оцениваем прогиб балки в точке $P = 750$.

```
error_estimate = h ** 4 * np.max(np.abs(cs(P_values, 4))) / 384
```

Оценка максимальной погрешности кубического сплайна:

$$error \approx \frac{h^4}{384} \cdot \max |f^{(4)}(P)|,$$

где:

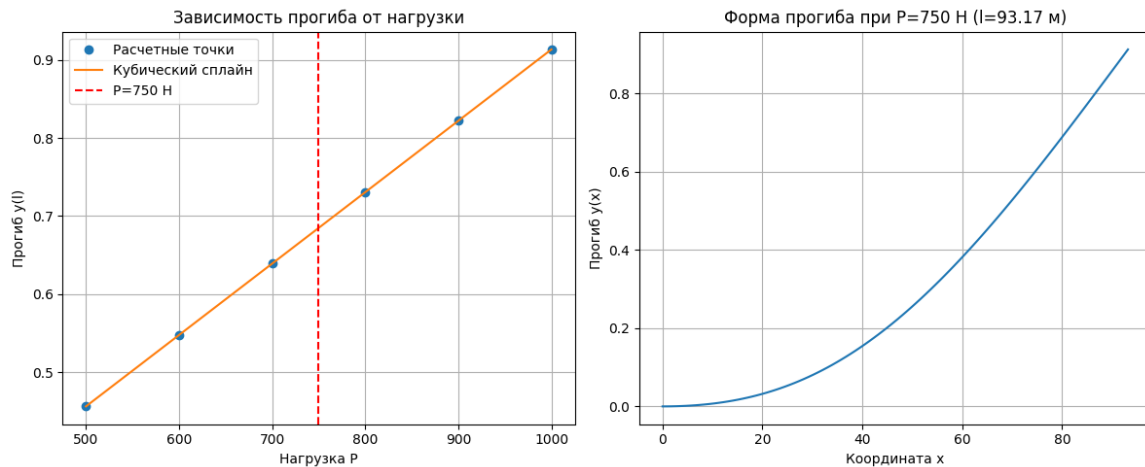
- h – шаг между точками (=100)
- $f^{(4)}$ – четвертая производная функции $y(l)$

Визуализация результатов:

С помощью библиотеки `matplotlib` строим 2 графика:

1. Зависимость прогиба от нагрузки
2. Форма прогиба балки

Вывод программы:



Найденный корень $x^* = 0.931678$

Длина балки $l = 93.17$

Результаты расчета:

P	$y(l)$
500	0.456590
600	0.547908
700	0.639227
800	0.730545
900	0.821863
1000	0.913181

Интерполированное значение при $P=750$: 0.684886 м

Оценка погрешности: 0.00e+00

Вывод:

Погрешность интерполяции оказалась нулевой, так как зависимость прогиба от нагрузки близка к линейной, а кубический сплайн точно воспроизводит эти данные, потому что наши расчетные точки лежат почти на одной линии. Это связано с тем, что первая и вторая производные линейно зависят от нагрузки P , также члены dI/dx и d^2I/dx^2 не сильно влияют на систему ввиду затухания по экспоненте $e^{-6x/l}$.

Погрешность была уже при вычислении исходной длины балки, также на погрешность повлияли численные ошибки интегрирования ОДУ, но их влияние минимально благодаря малым допускам относительной и абсолютной погрешностей $rtol=1e-6$ и $atol=1e-8$.